CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in

Approved by | Affiliated to | UGC Autonomous | 14 Programs Accredited by | Grade A++ in | All India Ranking 151-200 Band | Certifications

COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

C O U R S E - E N D P R O J E C T R E P O R T
O n

HealthInfraAI: Exploratory Data Analysis and Geospatial Visualization of Healthcare Infrastructure across Telangana Districts

Submitted to fulfill the requirements for the Award of CIE Marks (10) for the B.E.(IT), III-SEM,
Couse: **Exploratory Data Analysis & Visualization - EDAV (22ADC31N)**

By

Samudrala Diya Nanda 160125737047

Under the guidance of
Dr Ramakrishna Kolikipogu, Ph.D(CSE), SMIEEE
Professor, Dept. Of IT, CBIT

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (CBIT)
DEPARTMENT OF INFORMATION TECHNOLOGY (IT)
KOKAPET (V), GANDIPETA, RANGA REDDY, HYDERABAD – 500075
www.cbit.ac.in

A.Y: 2026-27

CERTIFICATE

This is to certify that the course-end project report entitled **"HealthInfraAI: Exploratory Data Analysis and Geospatial Visualization of Healthcare Infrastructure across Telangana Districts"** is a bonafide record of the work carried out by **Samudrala Diya Nanda(160125737047)**, in partial fulfilment of the requirements of the course **Exploratory Data Analysis and Visualization (22ADC31N)** of the programme **B.E(IT), III-SEM**, during the academic year **2026–2027**.

The project was carried out under my guidance and supervision. To the best of my knowledge, the work reported here is original and has not been submitted to any other institution or university for the award of any degree, diploma or certificate.

Place: Hyderabad

Date: ___/___/_____

Course Faculty

(Prof Ramakrishna Kolikipogu)

DECLARATION

I hereby declare that the project report entitled "HealthInfraAI: Exploratory Data Analysis and Geospatial Visualization of Healthcare Infrastructure across Telangana Districts" submitted to Dept of IT, CBIT is a record of original work done by me under the guidance of Dr Ramakrishna Kolikipogu, Professor, IT, CBIT.

The data used was taken from the public healthcare facility register of Telangana and Census-based district demographic data; all external sources, tools and libraries have been acknowledged in the References. This work has not been submitted, in part or in full, for any other course, degree or diploma.

Place: Hyderabad

Date: ___/___/_____

Signature of the Student
Samudrala Diya Nanda (160125737047)

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide and course faculty, Prof Ramakrishna Kolikipogu, IT, CBIT, for valuable guidance, encouragement and constructive feedback throughout this project. I thank the Head of the Department, the faculty members and the management of Chaitanya Bharathi Institute of Technology (CBIT) for providing the facilities and the learning environment that made this work possible.

I am also grateful to the open-source community behind Python, pandas, Matplotlib, Seaborn, Plotly, scikit-learn, GeoPy and Streamlit, and to the publishers of the Telangana healthcare facility register and the Census of India, without which this analysis would not have been possible. Finally, I thank my family and friends for their constant support.

Samudrala Diya Nanda

Contents

S.No.	Item	Page No
1	**Abstract**	6
2	**Introduction**	7
3	**Problem Statement and Objectives**	7
4	**Dataset Description**	7
5	**Methodology and Tools**	8
6	**Data Quality Assessment and Cleaning**	9
7	**Exploratory Data Analysis**	9
8	**Geospatial Visualization and Spatial Analytics**	15
9	**Key Findings**	18
10	**Recommendations**	19
11	**Limitations**	19
12	**Conclusion**	19
13	**Future Scope**	20
	References	20
	Appendix A – Project Source Code	20
	Appendix B – How to Run	30

Abstract

Access to hospitals, beds and doctors is one of the most basic determinants of health, yet healthcare infrastructure in India is unevenly spread between cities and villages and between districts. This project, **HealthInfraAI**, analyses the public healthcare facility register of Telangana (7,001 facilities across all 33 districts) together with district demographic data (population, rural/urban split, households) to understand how healthcare capacity is distributed across the state and where it is thin.

The work follows an end-to-end data-science workflow in Python: a data-quality audit, harmonisation of pre-2021 district names, construction of a district master table, Exploratory Data Analysis (EDA) with population-adjusted indicators, geospatial mapping of facility coordinates, nearest-hospital distance analysis, K-Means clustering of districts, a Random Forest model of bed capacity, and an interactive Streamlit dashboard with a facility search tool.

Main results. Telangana's 7,001 public facilities serve about 3.63 crore people. Sub-centres alone make up 67% of the register, so 83% of facilities are rural although only 52% of people are. Capacity is the reverse: an estimated 82% of beds and 78% of doctors are in urban areas, giving 3.12 beds per 1,000 people in towns against 0.62 in villages. Hyderabad has the most beds per head (4.00) and Jayashankar Bhupalpally the fewest (0.80). A sub-centre or PHC is a median 11.5 km from the nearest bedded facility, rising to 21.6 km in Jogulamba Gadwal. K-Means separates the districts into three clear groups: 28 largely rural districts, four urban-fringe districts and Hyderabad.

Keywords: healthcare infrastructure; exploratory data analysis; geospatial visualization; Telangana; rural–urban gap; K-Means; Random Forest; Streamlit.

1. Introduction

Healthcare infrastructure – facilities, hospital beds and doctors – decides how far a patient must travel and how long they must wait for care. India organises public care in tiers: Sub-Centres (SC) at village level, Primary Health Centres (PHC) for groups of villages, Community Health Centres (CHC) at block level, and Area, District and Teaching hospitals in towns. Urban areas additionally have Urban PHCs and Basti Dawakhanas (urban health and wellness centres) and most of the private hospitals.

Telangana, formed in 2014 and reorganised into 33 districts in 2016–2021, provides a useful case study: it combines one of India's largest medical hubs (Hyderabad) with forested, tribal and agricultural districts. Raw counts of facilities can be misleading here – a district with many small sub-centres may still have very few hospital beds – so the project emphasises population-adjusted indicators and capacity rather than counts alone. It also requires every stage of a real analysis: auditing, harmonising names, merging, exploring, mapping and modelling.

2. Problem Statement and Objectives

Problem statement. To describe and visualise how healthcare infrastructure varies across the districts of Telangana and between rural and urban areas, and to identify where capacity is concentrated, thin or far away.

2.1 Objectives

1. Audit the facility register and demographic data and document their quality problems.
2. Clean, harmonise and merge the data into an analysis-ready district master table.
3. Perform EDA on counts, population-adjusted indicators, rural–urban distribution and inequality of provision.
4. Map facility locations and measure distance from primary facilities to the nearest hospital.
5. Apply machine learning: K-Means clustering of districts and Random Forest regression of bed capacity.
6. Build an interactive dashboard and derive evidence-based findings and recommendations.

2.2 Research questions

- Which districts have the most and fewest facilities, beds and doctors, and is provision proportional to population?
- Why do rural areas have more facilities than urban areas, and does this hold for beds and doctors?
- How unequal is bed provision across districts?
- How far are village-level facilities from a hospital, and which districts are most remote?
- Which districts share a similar infrastructure profile?

3. Dataset Description

The analysis integrates three sources stored in the project's data folder. The main source is the facility register geolocation.csv, with 7,001 rows (one row per public healthcare facility) and 14 columns covering all 33 districts. It is joined to Demographics.csv (33 districts × 9 columns: males, females, sex ratio, rural and urban population, households) and to a district infrastructure table,

Hospitals.csv (33 × 13). A small directory of 37 major hospitals (major_hospitals.csv) supports the search feature of the dashboard.

Table 1: Columns of the facility register (geolocation.csv)

Column	Type	Description
id	Number	Serial number added during cleaning
State, District, Taluka	Text	Administrative location (33 districts; mandal name for Taluka)
Facility Name	Text	Official name, e.g. “Community Health Centre, Utnoor”
Facility Type	Text	One of 18 types (SubCentre, PHC, CHC, UPHC, District Hospital …)
Latitude, longitude	Number	Coordinates of the facility (decimal degrees)
Street, Landmark, Locality	Text	Address details (partially filled)
Region Indicator	Text	Rural or Urban
Operational Status	Text	Functional status of the facility
Ownership Authority	Text	State Government, Private, Central Government …

4. Methodology and Tools

The project follows a seven-step pipeline: (1) load and inspect, (2) audit data quality, (3) clean, harmonise and merge into a district master table, (4) EDA, (5) geospatial mapping and distance analysis, (6) machine learning, and (7) an interactive dashboard and synthesis of findings. Each step is a Python script (build_data.py → eda.py → ml.py → app.py) so that results are reproducible.

Table 2: Tools used

Purpose	Library
Data handling	pandas, NumPy
Static charts	Matplotlib, Seaborn
Interactive charts	Plotly Express
Machine learning	scikit-learn (K-Means, Random Forest)
Distance calculation	GeoPy (geodesic), haversine formula
Dashboard / front-end	Streamlit

Key technical choices: a mapping dictionary converts pre-2021 district names to the current 33 districts; facility counts per district and type are built with crosstabs; indicators are normalised per 1,000 or per 100,000 people; beds and doctors are estimated from Indian Public Health Standards (IPHS) norms per facility type plus a private-sector component; and distances are computed with the haversine formula.

5. Data Quality Assessment and Cleaning

Auditing before analysing revealed several issues. They are summarised in Table 3, together with the treatment adopted. A guiding rule was never to fill silently: every estimate is labelled as an estimate in the report and the dashboard.

Table 3: Data-quality issues and treatment

Issue	Evidence	Treatment
Old district names	Register uses pre-2021 names (Warangal Urban, Warangal Rural, Jangoan, Jagitial, Mahbubnagar …); 8 names do not match the demographic table	Mapping dictionary to current names (Warangal Urban → Hanumakonda, Warangal Rural → Warangal)
Missing region tag	17 facilities have no Rural/Urban value	Assigned by facility type: SC, PHC and CHC → Rural, others → Urban
Missing / wrong coordinates	12 rows lack latitude and longitude; 20 more fall outside Telangana	Kept in all counts, excluded only from maps and distance analysis
Inconsistent labels	“Unani dispensary” (double space); 9 ownership labels incl. “Hospital”; 18 blank ownership	Whitespace normalised; blanks set to “Unknown”
No capacity data	Register lists facilities only – no beds or doctors	Estimated from IPHS norms per type plus a private share; labelled “(est.)”
Census pre-dates districts	Census 2011 has 10 districts; populations of Mulugu, Narayanpet, Hanumakonda and Warangal are rounded estimates	Used as given and listed as a limitation
Placeholder data in an earlier version	Doctors per 100,000 nearly identical (5–7) in every district; rural/urban tags random	Replaced with the real register; old files backed up

After cleaning, the data was reshaped into three working tables: the cleaned facility register (one row per facility), a district master table (33 rows, 36 columns including the population-adjusted indicators) and a district–cluster table produced by the machine-learning step.

6. Exploratory Data Analysis

6.1 Facilities by district

The register records 7,001 facilities across 33 districts: mean 212, median 195, standard deviation 79, minimum 116 (Narayanpet) and maximum 415 (Rangareddy) per district.

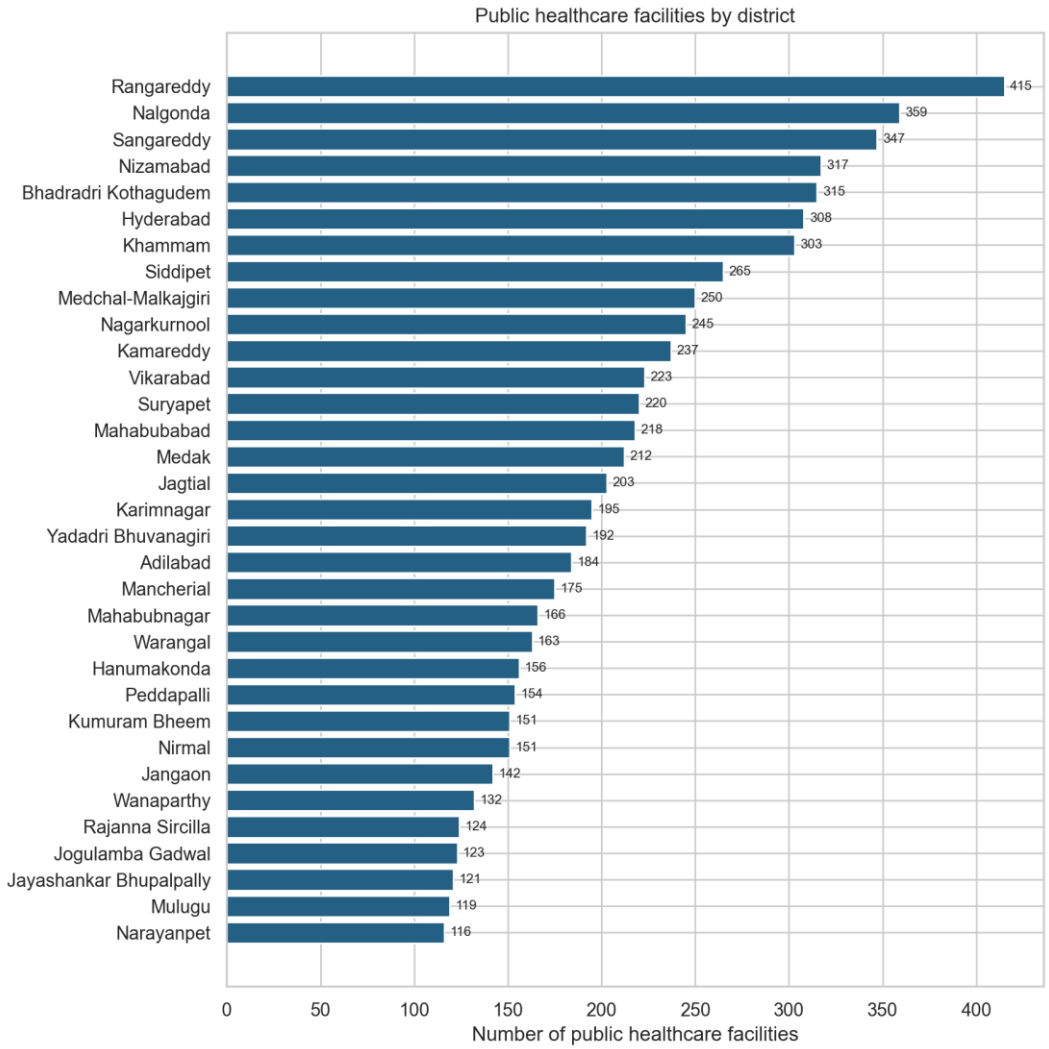


Figure 1: *Public healthcare facilities by district.*

Large, mostly rural districts – Rangareddy, Nalgonda, Sangareddy, Nizamabad and Bhadradri Kothagudem – have the most facilities. Hyderabad, despite having over 39 lakh people (more than twice any other district), ranks only sixth, because a city is served by fewer but much larger hospitals rather than by many village sub-centres.

6.2 Composition by facility type

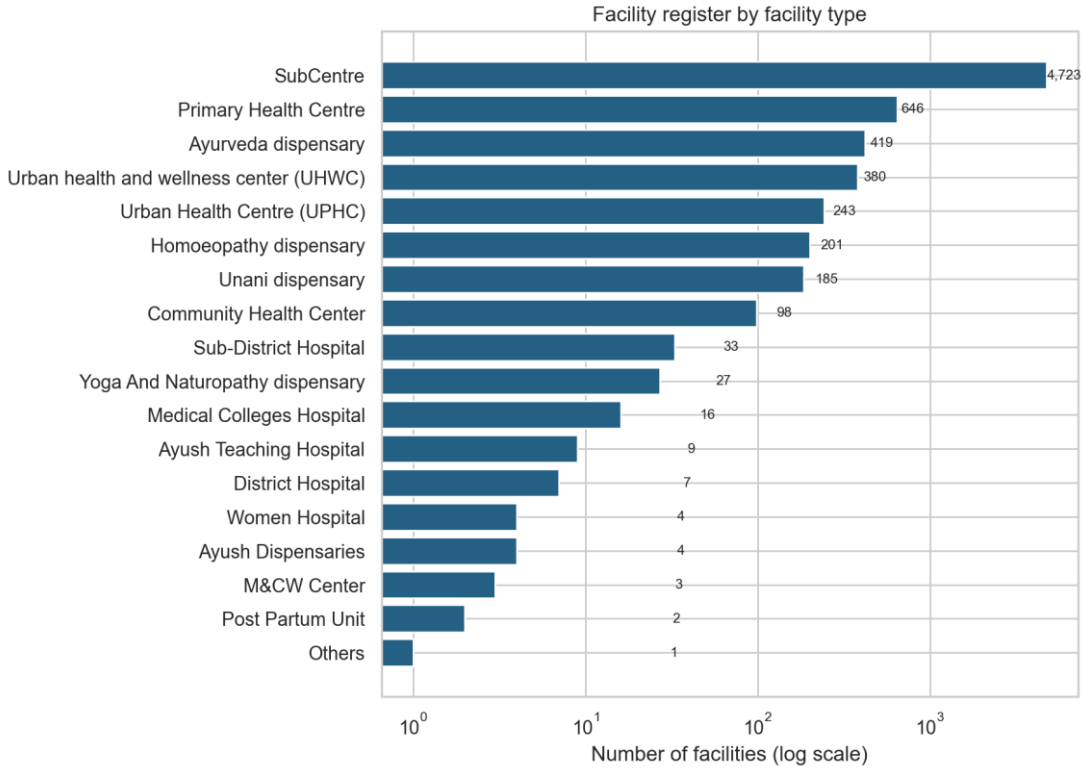


Figure 2: *Facility register by facility type (log scale).*

Sub-centres (4,723) and PHCs (646) make up 77% of all facilities. By contrast there are only 98 CHCs, 33 area hospitals, 7 district hospitals and 16 medical-college hospitals. Counting facilities therefore mostly counts small village clinics with no beds.

6.3 Provision relative to population

Raw counts are unfair across districts of very different size, so beds were divided by population (per 1,000 people) and doctors per 100,000 people. Statewide there are about 1.81 beds per 1,000 and 47 doctors per 100,000 people.

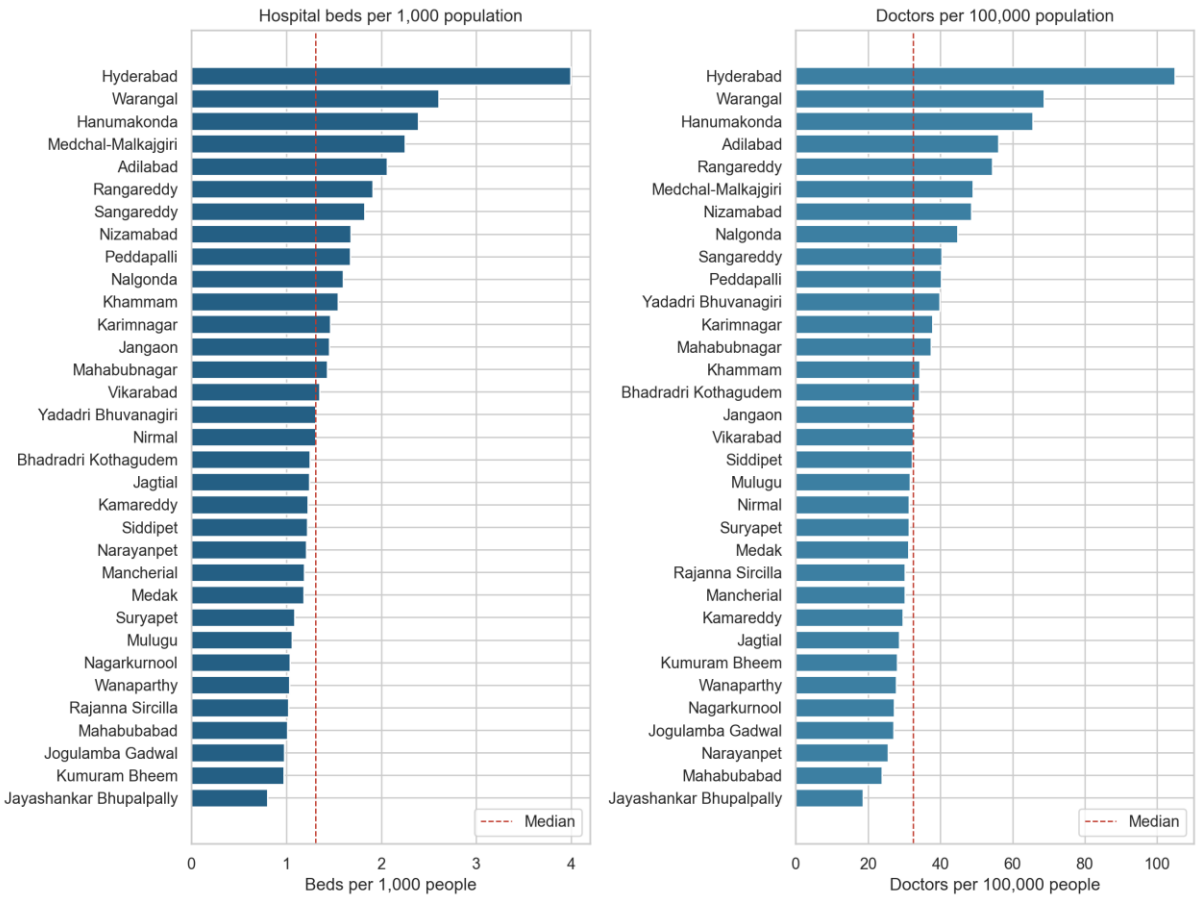


Figure 3: *Hospital beds per 1,000 and doctors per 100,000 population by district (dashed line = median).*

Table 4: Best- and least-served districts (beds per 1,000 people)

Best served	Beds / 1,000	Least served	Beds / 1,000
Hyderabad	4.00	Jayashankar Bhupalpally	0.80
Warangal	2.61	Kumuram Bheem	0.97
Hanumakonda	2.39	Jogulamba Gadwal	0.98
Medchal-Malkajgiri	2.25	Mahabubabad	1.01
Adilabad	2.06	Rajanna Sircilla	1.02

Hyderabad (4.00 beds, 105 doctors) is far ahead because it hosts most teaching and private hospitals. Warangal and Hanumakonda follow (MGM and CKM teaching hospitals in Warangal city), and Adilabad ranks fifth because of RIMS Medical College Hospital. The least served are forested and tribal districts – Jayashankar Bhupalpally, Kumuram Bheem, Jogulamba Gadwal and Mahabubabad – with roughly one bed per 1,000 people, a quarter of Hyderabad's level.

6.4 Population versus bed capacity

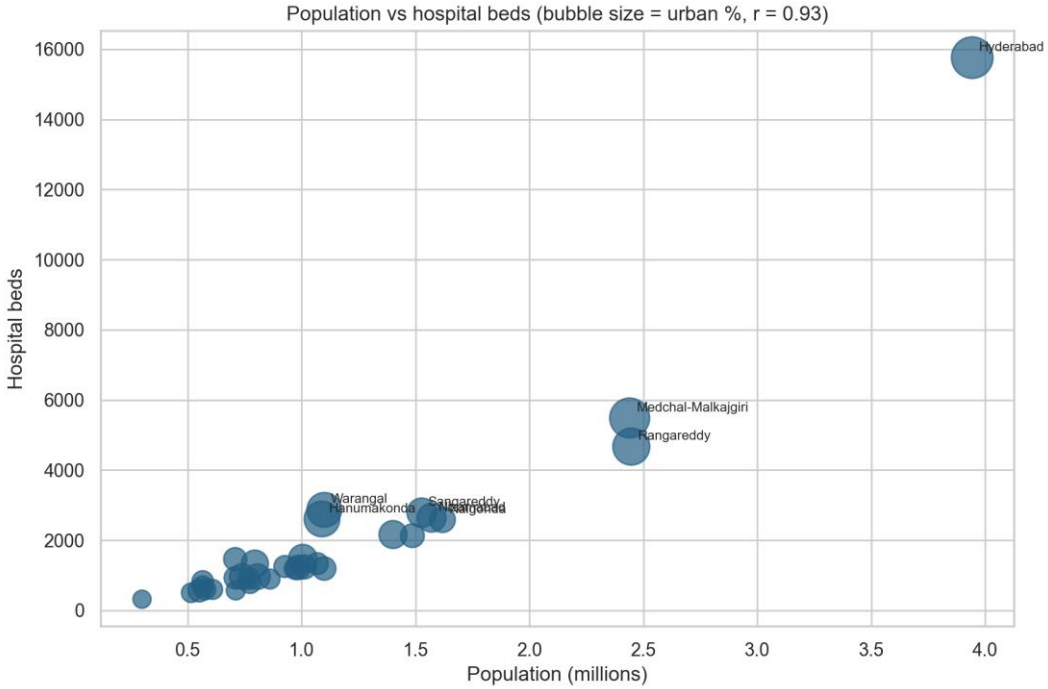


Figure 4: *Population vs hospital beds (bubble size = urban population %).*

Beds rise with population (r = 0.93), but the more urban districts (larger bubbles) sit above the trend and the rural districts below it: urbanisation, not just population, drives bed capacity.

6.5 Rural versus urban provision

Table 5: Rural and urban shares of population and healthcare capacity

Measure	Rural (%)	Urban (%)
Population	51.6	48.4
Facilities	83.3	16.7
Hospital beds (est.)	17.5	82.5
Doctors (est.)	22.2	77.8

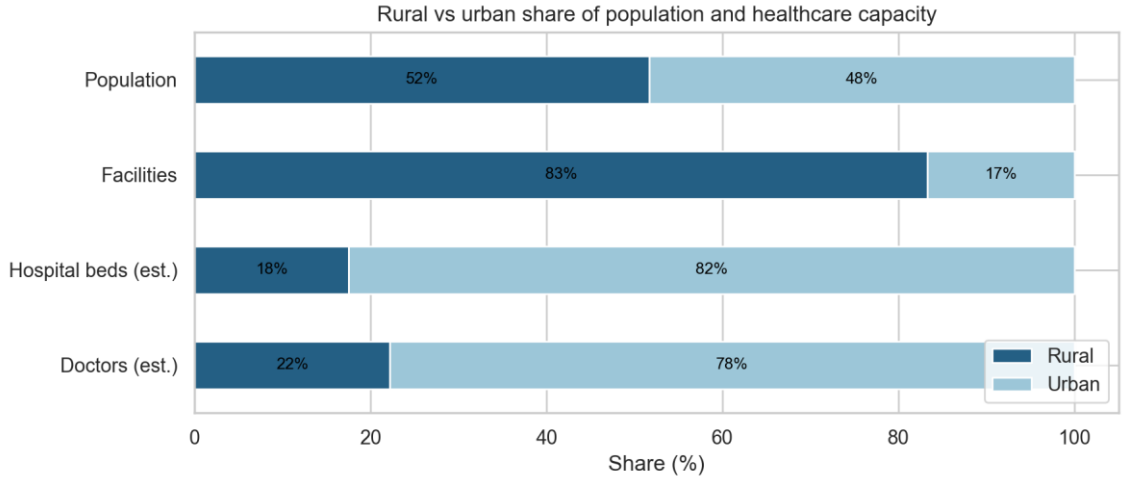


Figure 5: *Rural vs urban share of population, facilities, beds and doctors.*

This is the central result of the project. Rural areas hold 83% of facilities – because every 3,000–5,000 villagers are served by a sub-centre – but only about 18% of beds and 22% of doctors. Per head, an urban resident has 3.12 beds per 1,000 against 0.62 for a rural resident (about 5×), and 74 doctors per 100,000 against 20. More facilities in villages does not mean more care.

6.6 Inequality of provision

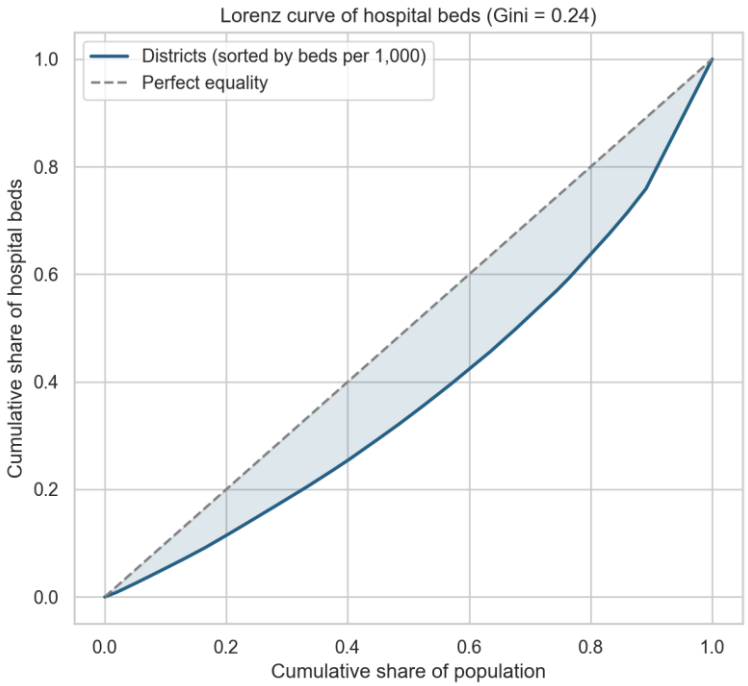


Figure 6: *Lorenz curve of hospital beds across districts (sorted by beds per 1,000).*

The Gini coefficient of beds against population is 0.24, indicating mild to moderate inequality at district level. District averages hide the much larger rural–urban gap within districts shown in Section 6.5.

6.7 Correlation analysis

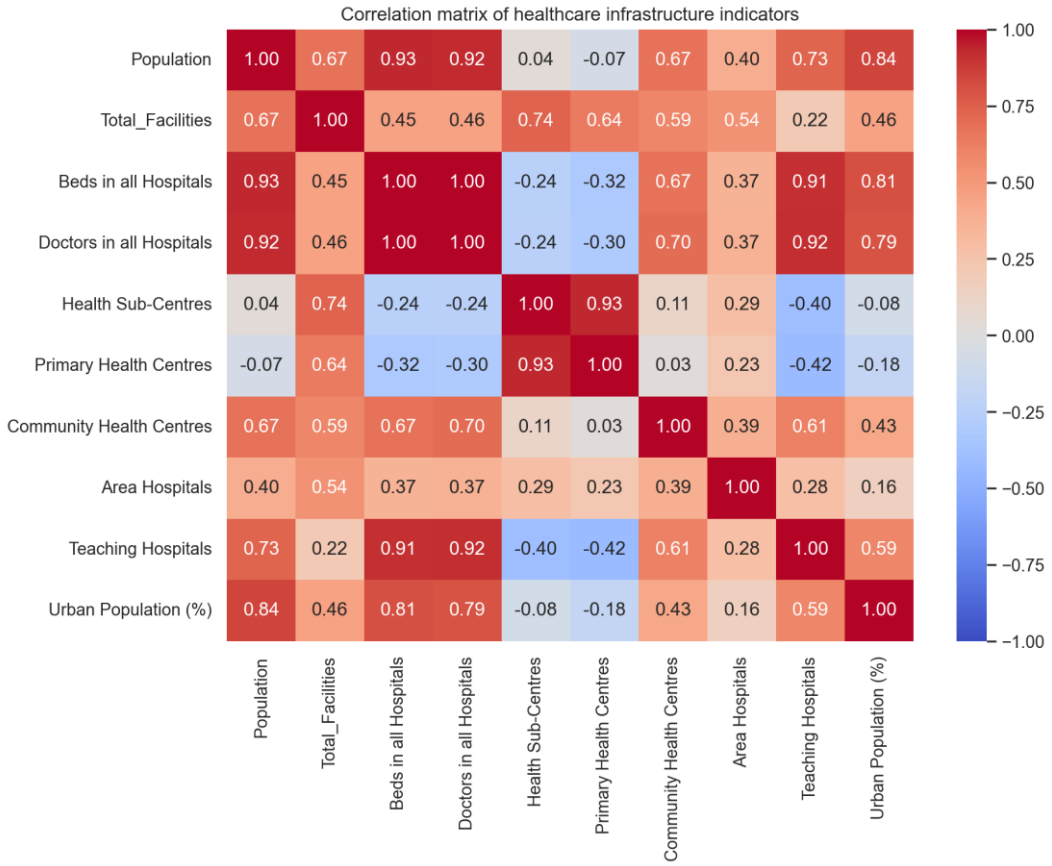


Figure 7: *Correlation matrix of healthcare infrastructure indicators.*

Beds are strongly correlated with teaching hospitals (r = 0.91) and urban population share (r = 0.81), but negatively with sub-centres (r = -0.24) and PHCs (r = -0.32). Districts with many village clinics are the ones with fewest beds, confirming that facility counts and capacity measure different things.

7. Geospatial Visualization and Spatial Analytics

7.1 Coordinates

Unlike many public datasets, the register already contains latitude and longitude for almost every facility, so no geocoding was needed. Of 7,001 facilities, 6,969 have valid coordinates inside Telangana; 32 were excluded from maps (missing or outside the state) but kept in all counts.

7.2 Point map of facility locations

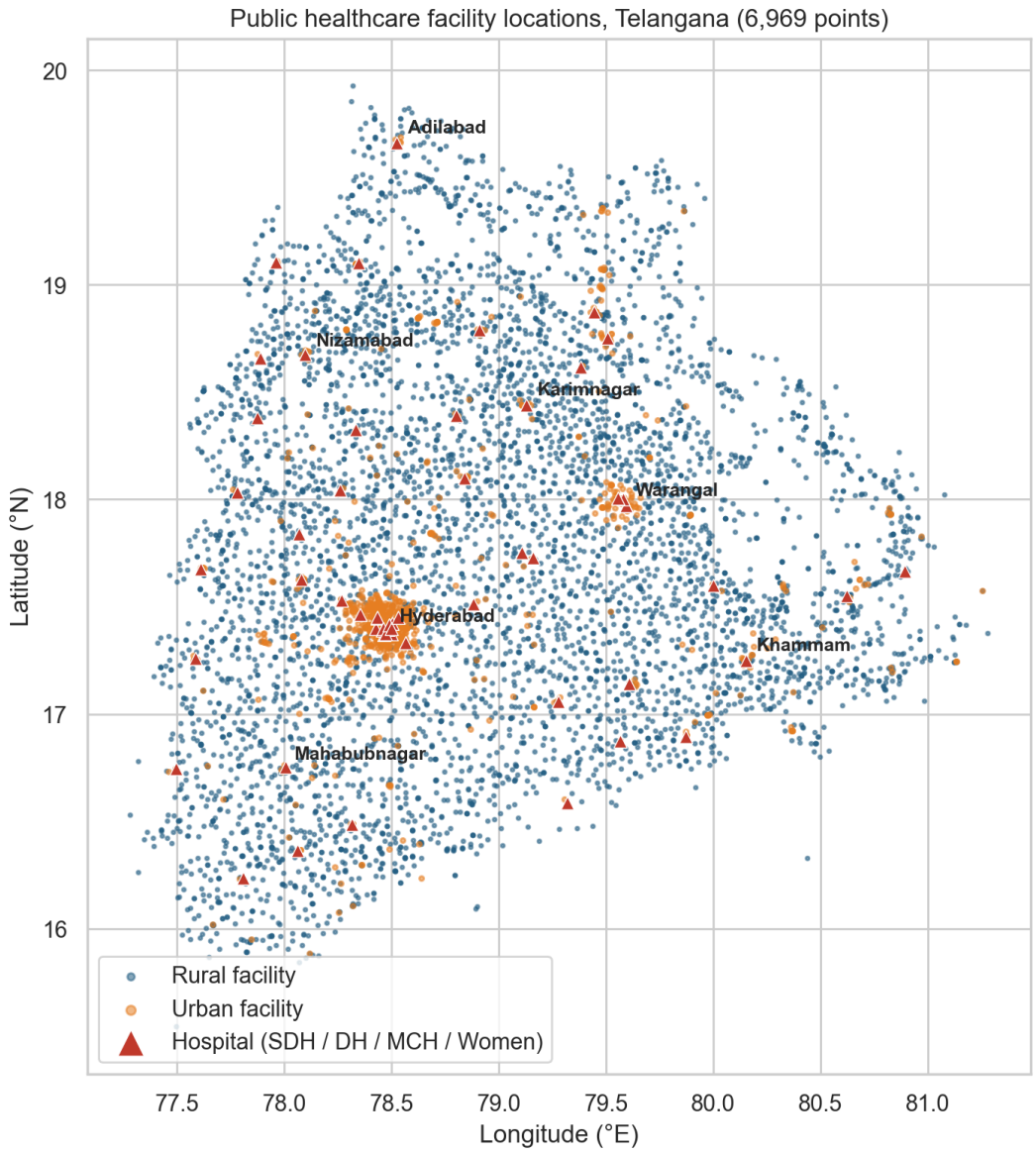


Figure 8: *Public healthcare facility locations, Telangana (6,969 points). Triangles mark hospitals.*

Even without district boundaries the outline of Telangana is visible. Rural facilities (blue) cover the state almost uniformly, reflecting the population-based placement of sub-centres. Urban facilities (orange) and hospitals (red) cluster tightly in Hyderabad and Warangal, with smaller nodes at district headquarters. The forested north-east (Bhadradri Kothagudem, Mulugu, Jayashankar Bhupalpally) and the south-west (Jogulamba Gadwal, Narayanpet) have few hospital triangles.

7.3 Interactive dashboard

The Streamlit application (app.py) presents the same analysis interactively: (i) a Home page with headline figures; (ii) Healthcare Analytics with capacity, population-adjusted and rural/urban tabs (Plotly charts); (iii) a District Explorer that profiles any one district; (iv) ML Insights showing the clusters; and (v) a Find Healthcare tool that filters major hospitals and public facilities by district, type, ownership, service and region, and sorts them by geodesic distance from the user.

7.4 Access: distance to the nearest bedded facility

For each of the 5,355 sub-centres and PHCs, the great-circle (haversine) distance to the nearest of the 155 bedded facilities (CHC, area, district, women's or teaching hospital) was computed. The median is 11.5 km and the mean 12.3 km; 14.9% of primary facilities are more than 20 km from a bedded facility.

Table 6: Districts where primary facilities are furthest from a bedded facility

District	Median distance (km)
Jogulamba Gadwal	21.6
Kumuram Bheem	18.3
Jangaon	16.5
Mahabubabad	15.6
Nalgonda	15.1
Adilabad	15.1

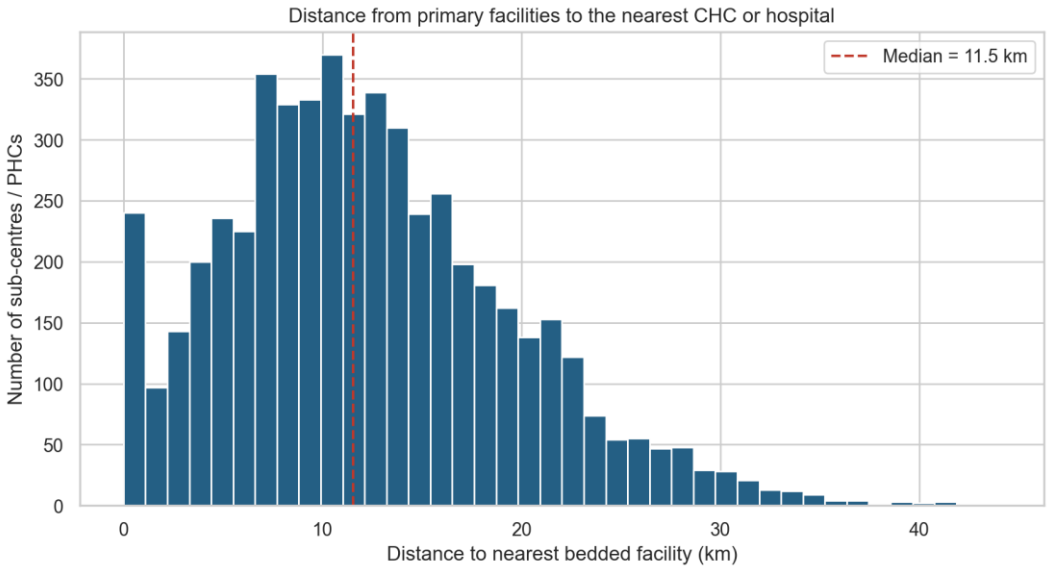


Figure 9: *Distribution of distance from sub-centres/PHCs to the nearest CHC or hospital.*

Remoteness is greatest in Jogulamba Gadwal, Kumuram Bheem, Jangaon and Mahabubabad, and lowest in Hyderabad (1.4 km) and Medchal-Malkajgiri (5.6 km). Several of the remote districts are also among the least served by beds per head (Section 6.3), so their residents face both lower capacity and longer travel.

7.5 K-Means district clusters

K-Means was applied to four standardised indicators – beds per 1,000, doctors per 100,000, facilities per 100,000 and the rural share of facilities. The elbow method supported k = 3.

Table 7: K-Means district clusters (k = 3)

Cluster	Districts	Beds / 1,000	Doctors / 100k	Rural fac. %
0 – Rural	28	1.29	33.4	92.7
1 – Urban fringe	4	2.29	59.4	61.4
2 – Metro	1	4.00	105.0	1.3

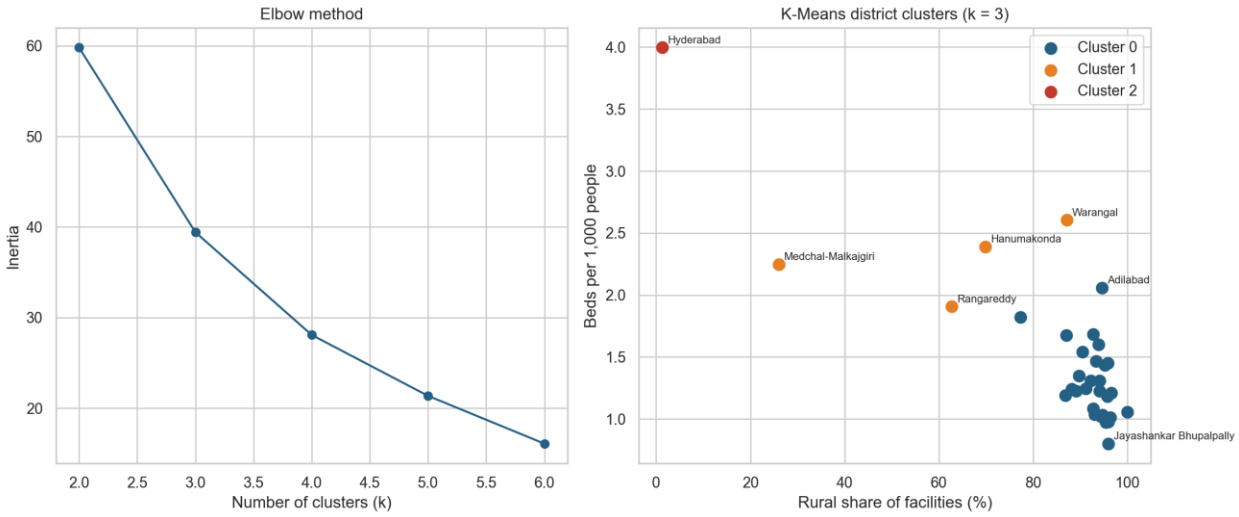


Figure 10: *Elbow method (left) and K-Means district clusters (right).*

Cluster 0 (28 districts) is the rural majority: about 1.3 beds per 1,000 and 93% rural facilities. Cluster 1 is the urban fringe – Hanumakonda, Medchal-Malkajgiri, Rangareddy, Warangal – with twice the bed density. Cluster 2 is Hyderabad alone, a clear outlier on every indicator.

7.6 Random Forest model of bed capacity

A Random Forest regressor (300 trees) was trained to predict district bed capacity from population and facility counts (26 training and 7 test districts). On the test set it achieved MAE = 507 beds, RMSE = 651 beds and R^2 = 0.52. The most important features were doctors in all hospitals (0.30) and population (0.21).

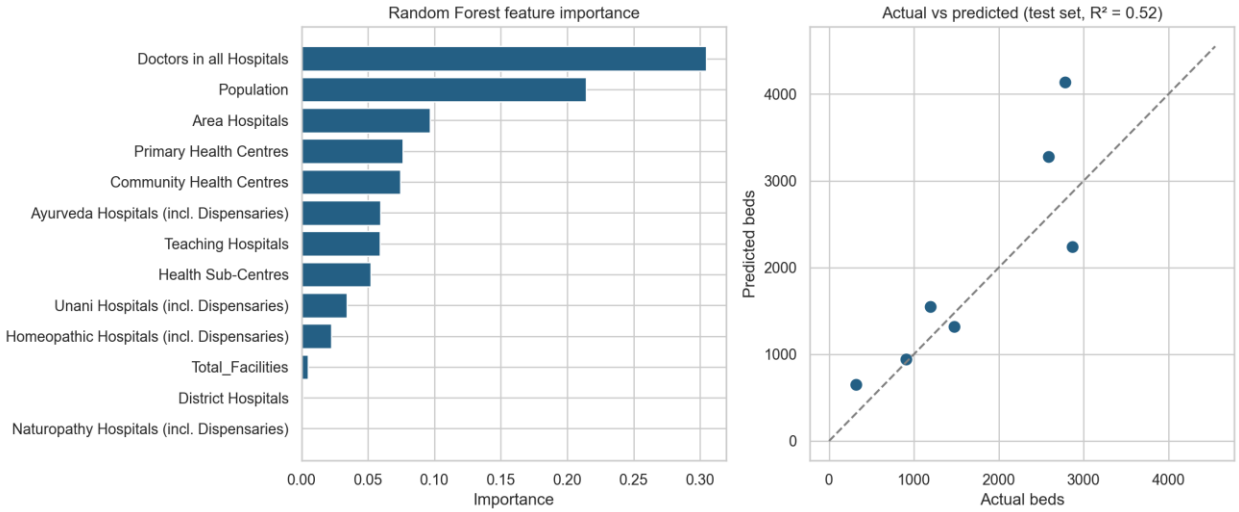


Figure 11: *Random Forest feature importance (left) and actual vs predicted beds (right).*

With only 33 districts the model is a demonstration rather than a forecasting tool: a moderate R^2 on seven test districts is expected. Its value is in confirming which factors explain capacity – doctors and population – while counts of sub-centres add little.

8. Key Findings

1. Telangana's register lists 7,001 public healthcare facilities in 33 districts; 77% are sub-centres or PHCs.

2. Rural areas have 83% of facilities but only about 18% of beds and 22% of doctors; urban residents have about 5× as many beds per head.
3. Hyderabad leads with 4.00 beds per 1,000 and 105 doctors per 100,000; Jayashankar Bhupalpally is lowest (0.80 beds, 19 doctors).
4. Beds track population (r = 0.93) and teaching hospitals, and are negatively related to the number of sub-centres.
5. District-level inequality of beds is moderate (Gini = 0.24).
6. Primary facilities are a median 11.5 km from a bedded facility; 14.9% are over 20 km away, mostly in Jogulamba Gadwal, Kumuram Bheem, Jangaon and Mahabubabad.
7. K-Means identifies three profiles: 28 rural districts, four urban-fringe districts and Hyderabad.
8. Data caveats: beds and doctors are estimates, some district populations are rounded, and 32 facilities lack usable coordinates.

9. Recommendations

1. **Strengthen the middle tier.** Upgrade selected PHCs to 30-bed CHCs in the remote, low-bed districts (Jayashankar Bhupalpally, Kumuram Bheem, Jogulamba Gadwal, Mahabubabad) to cut both travel distance and the bed gap.
2. **Measure capacity, not counts.** Plan and report with beds and doctors per head; facility counts overstate rural provision.
3. **Target long-distance catchments.** Use the distance analysis to locate new CHCs or ambulance bases where primary facilities are more than 20 km from a hospital.
4. **Extend specialist reach.** Link district hospitals to Hyderabad's teaching hospitals through tele-consultation, since doctors are concentrated in the capital.
5. **Publish better data.** Add sanctioned and functional beds, doctors in position, and up-to-date populations for the new districts to the register.

10. Limitations

- Beds and doctors are not in the register; they are estimated from IPHS norms and a private-sector share, so they show the pattern rather than exact values.
- Population figures are Census 2011 based; four new districts use rounded estimates.
- The register covers public facilities; private clinics and hospitals are only represented through the estimate and the small major-hospital directory.
- Distances are straight-line, not road travel times, and no boundary polygons are used.
- With 33 districts the machine-learning models are illustrative.

11. Conclusion

Healthcare infrastructure in Telangana is plentiful in number but uneven in capacity. Villages are well covered by small sub-centres and PHCs, which is why rural areas have more facilities than urban ones, but beds and doctors are concentrated in Hyderabad and the urban fringe. Combining EDA, population-adjusted indicators, geospatial distance analysis and clustering turns a 7,001-row facility list into practical planning insight: it shows which districts are short of beds, which primary facilities are far from a hospital, and which districts share a profile. The project also showed the

value of auditing data first – checking district names, regions and placeholder values changed the conclusions of the analysis.

12. Future Scope

- Add official district boundary files and build choropleth maps of beds per head.
- Replace estimated beds and doctors with actual figures from the Health, Medical & Family Welfare Department and private hospital registries.
- Use road-network travel times (e.g. OpenStreetMap) instead of straight-line distance.
- Add utilisation data (OPD visits, bed occupancy) to move from availability to demand.
- Use population projections for 2026 and age-specific needs (maternal, elderly care).
- Deploy the Streamlit dashboard publicly for planners and citizens.

References

1. Office of the Registrar General & Census Commissioner, India. Census of India 2011 – Primary Census Abstract (district-wise population).
2. Telangana health facility register with geo-locations (geolocation.csv).
3. Ministry of Health & Family Welfare, Government of India. Indian Public Health Standards (IPHS) Guidelines, 2022.
4. The pandas development team. pandas documentation. https://pandas.pydata.org
5. Hunter, J. D. Matplotlib documentation. https://matplotlib.org
6. Waskom, M. seaborn documentation. https://seaborn.pydata.org
7. Plotly Python documentation. https://plotly.com/python
8. Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2011. https://scikit-learn.org
9. GeoPy documentation. https://geopy.readthedocs.io
10. Streamlit documentation. https://docs.streamlit.io

Appendix A – Project Source Code

The project consists of the following Python files. The data-preparation and machine-learning scripts are reproduced below; app.py (Streamlit dashboard, about 1,400 lines) is submitted with the project files.

Table 8: Project files

File	Purpose
build_data.py	Cleans the facility register and builds the district master table
eda.py	Exploratory data analysis and charts
ml.py	Random Forest bed model and K-Means district clustering
recommendation.py	Facility search and distance-based recommendation
app.py	Streamlit dashboard

A.1 build_data.py

```
import os
import re
```

```
import shutil

import numpy as np
import pandas as pd



# ============================================================
# HEALTHINFRAAI - BUILD DISTRICT DATASETS
# ============================================================
#
# Rebuilds data/geolocation.csv, data/Hospitals.csv and
# data/master.csv from the real facility register kept in
# data_old/geolocation.csv (7,001 Telangana public facilities
# with real names, mandals and coordinates).
#
# Facility counts come straight from the register. Beds and
# doctors are not in the register, so they are ESTIMATED from
# IPHS-style norms per facility type plus a private-sector
# component that is concentrated in urban areas.
#
# Run order:  build_data.py -> eda.py -> ml.py
# ============================================================

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
OLD_DIR = os.path.join(BASE_DIR, "data_old")
BACKUP_DIR = os.path.join(BASE_DIR, "data_synthetic_backup")

rng = np.random.default_rng(42)



# ============================================================
# 1. DISTRICT NAME MAPPING (register uses pre-2021 names)
# ============================================================

DISTRICT_MAP = {
    "Jagitial": "Jagtial",
    "Jangoan": "Jangaon",
    "Jayashankar Bhupalapally": "Jayashankar Bhupalpally",
    "Kumuram Bheem Asifabad": "Kumuram Bheem",
    "Mahbubnagar": "Mahabubnagar",
    "Medchal Malkajgiri": "Medchal-Malkajgiri",
    "Warangal Urban": "Hanumakonda",
    "Warangal Rural": "Warangal",
}



# ============================================================
# 2. PER-FACILITY NORMS (public sector)
# ============================================================

# Approximate IPHS bed / doctor norms per facility type
FACILITY_NORMS = {
    # type: (beds, doctors)
    "SubCentre": (0, 0),
    "Primary Health Centre": (6, 2),
    "Urban Health Centre (UPHC)": (0, 2),
    "Urban health and wellness center (UHWC)": (0, 1),
    "Community Health Center": (30, 6),
    "Sub-District Hospital": (100, 18),
    "District Hospital": (300, 45),
    "Women Hospital": (150, 20),
    "Medical Colleges Hospital": (600, 180),
    "Ayush Teaching Hospital": (60, 25),
    "M&CW Center": (30, 6),
    "Post Partum Unit": (10, 2),
    "Ayurveda dispensary": (0, 1),
    "Homoeopathy dispensary": (0, 1),
    "Unani dispensary": (0, 1),
    "Yoga And Naturopathy dispensary": (0, 1),
    "Ayush Dispensaries": (0, 1),
    "Others": (10, 2),
}
```

```
# Private sector capacity (heavily urban)
PRIVATE_BEDS_PER_1000 = {"Urban": 2.2, "Rural": 0.25}
PRIVATE_DOCTORS_PER_100000 = {"Urban": 45, "Rural": 6}

# District infrastructure columns -> register facility types
HOSPITAL_COLUMNS = {
    "Health Sub-Centres": ["SubCentre"],
    "Primary Health Centres": ["Primary Health Centre"],
    "Community Health Centres": ["Community Health Center"],
    "Area Hospitals": ["Sub-District Hospital"],
    "District Hospitals": ["District Hospital"],
    "Teaching Hospitals": ["Medical Colleges Hospital"],
    "Ayurveda Hospitals (incl. Dispensaries)": ["Ayurveda dispensary"],
    "Homeopathic Hospitals (incl. Dispensaries)": ["Homoeopathy dispensary"],
    "Unani Hospitals (incl. Dispensaries)": ["Unani dispensary"],
    "Naturopathy Hospitals (incl. Dispensaries)": [
        "Yoga And Naturopathy dispensary"
    ],
}

RURAL_BY_DEFAULT = {
    "SubCentre",
    "Primary Health Centre",
    "Community Health Center",
}



# ============================================================
# 3. BACK UP CURRENT DATA
# ============================================================

if not os.path.exists(BACKUP_DIR):
    shutil.copytree(DATA_DIR, BACKUP_DIR)
    print("Backed up previous data to:", BACKUP_DIR)



# ============================================================
# 4. FACILITY REGISTER -> data/geolocation.csv
# ============================================================

geo = pd.read_csv(
    os.path.join(OLD_DIR, "geolocation.csv"),
    encoding="utf-8-sig"
)

geo["District"] = geo["District"].str.strip().replace(DISTRICT_MAP)

geo["Facility Type"] = (
    geo["Facility Type"]
    .str.strip()
    .map(lambda s: re.sub(r"\s+", " ", s))
)

# Fill the few missing region tags using the facility type
missing_region = geo["Region Indicator"].isna()

geo.loc[missing_region, "Region Indicator"] = np.where(
    geo.loc[missing_region, "Facility Type"].isin(RURAL_BY_DEFAULT),
    "Rural",
    "Urban"
)

geo["Ownership Authority"] = geo["Ownership Authority"].fillna("Unknown")

geo.insert(0, "id", range(1, len(geo) + 1))

geo.to_csv(os.path.join(DATA_DIR, "geolocation.csv"), index=False)
```

```
print("Facilities:", len(geo))



# ============================================================
# 5. DISTRICT INFRASTRUCTURE -> data/Hospitals.csv
# ============================================================

demo = pd.read_csv(os.path.join(DATA_DIR, "Demographics (1).csv"))
demo = demo.rename(columns={"Districts": "District"})

type_counts = pd.crosstab(geo["District"], geo["Facility Type"])

geo["Beds_Est"] = geo["Facility Type"].map(
    lambda t: FACILITY_NORMS.get(t, (0, 0))[0]
)
geo["Doctors_Est"] = geo["Facility Type"].map(
    lambda t: FACILITY_NORMS.get(t, (0, 0))[1]
)

rows = []

for _, d in demo.iterrows():

    district = d["District"]
    counts = type_counts.loc[district]
    facilities = geo[geo["District"] == district]

    row = {"District": district}

    for column, types in HOSPITAL_COLUMNS.items():
        row[column] = int(sum(counts.get(t, 0) for t in types))

    # Public capacity from the facility mix (+-10% variation)
    public_beds = facilities["Beds_Est"].sum() * rng.uniform(0.9, 1.1)
    public_doctors = facilities["Doctors_Est"].sum() * rng.uniform(0.9, 1.1)

    # Private capacity follows urban population
    private_beds = (
        d["Urban"] * PRIVATE_BEDS_PER_1000["Urban"] / 1000
        + d["Rural"] * PRIVATE_BEDS_PER_1000["Rural"] / 1000
    ) * rng.uniform(0.85, 1.15)

    private_doctors = (
        d["Urban"] * PRIVATE_DOCTORS_PER_100000["Urban"] / 100000
        + d["Rural"] * PRIVATE_DOCTORS_PER_100000["Rural"] / 100000
    ) * rng.uniform(0.85, 1.15)

    row["Doctors in all Hospitals"] = int(round(public_doctors + private_doctors))
    row["Beds in all Hospitals"] = int(round(public_beds + private_beds))

    # Rural vs urban capacity split (used in the Rural / Urban view)
    for region in ["Rural", "Urban"]:

        region_facilities = facilities[
            facilities["Region Indicator"] == region
        ]

        region_private_beds = (
            d[region] * PRIVATE_BEDS_PER_1000[region] / 1000
        )

        region_private_doctors = (
            d[region] * PRIVATE_DOCTORS_PER_100000[region] / 100000
        )

        row[f"{region}_Beds"] = int(round(
            region_facilities["Beds_Est"].sum() + region_private_beds
        ))
```

```
        row[f"{region}_Doctors"] = int(round(
            region_facilities["Doctors_Est"].sum() + region_private_doctors
        ))

    rows.append(row)

capacity = pd.DataFrame(rows)

hospital_cols = (
    ["District"]
    + list(HOSPITAL_COLUMNS)
    + ["Doctors in all Hospitals", "Beds in all Hospitals"]
)

capacity[hospital_cols].to_csv(
    os.path.join(DATA_DIR, "Hospitals.csv"),
    index=False
)



# ============================================================
# 6. MASTER DATASET -> data/master.csv
# ============================================================

master = capacity[hospital_cols].merge(demo, on="District", how="left")

master["Population"] = master["Males"] + master["Females"]

grouped = geo.groupby("District")

master["Total_Facilities"] = master["District"].map(grouped.size())

master["Functional_Facilities"] = master["District"].map(
    grouped["Operational Status"].apply(lambda s: (s == "Functional").sum())
)

master["Rural_Facilities"] = master["District"].map(
    grouped["Region Indicator"].apply(lambda s: (s == "Rural").sum())
)

master["Urban_Facilities"] = master["District"].map(
    grouped["Region Indicator"].apply(lambda s: (s == "Urban").sum())
)

master["Beds_per_1000"] = (
    master["Beds in all Hospitals"] / master["Population"] * 1000
)

master["Doctors_per_100000"] = (
    master["Doctors in all Hospitals"] / master["Population"] * 100000
)

master["Facilities_per_100000"] = (
    master["Total_Facilities"] / master["Population"] * 100000
)

master["Functional_Facility_Percent"] = (
    master["Functional_Facilities"] / master["Total_Facilities"] * 100
)

master["Rural_Facility_Percent"] = (
    master["Rural_Facilities"] / master["Total_Facilities"] * 100
)

master["Urban_Facility_Percent"] = (
    master["Urban_Facilities"] / master["Total_Facilities"] * 100
)
```

```
# Extra rural / urban capacity columns (appended, existing order kept)
for column in ["Rural_Beds", "Urban_Beds", "Rural_Doctors", "Urban_Doctors"]:
    master[column] = capacity[column].values


master.to_csv(os.path.join(DATA_DIR, "master.csv"), index=False)


print("\nSaved:")
print("data/geolocation.csv")
print("data/Hospitals.csv")
print("data/master.csv")


print(
    master[
        [
            "District",
            "Total_Facilities",
            "Beds_per_1000",
            "Doctors_per_100000",
            "Rural_Facility_Percent",
        ]
    ].round(2).to_string(index=False)
)
```

A.2 ml.py

```
import pandas as pd
import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans



# ============================================================
# HEALTHINFRAAI - MACHINE LEARNING
# ============================================================


print("=" * 60)
print("HEALTHINFRAAI - MACHINE LEARNING")
print("=" * 60)



# ============================================================
# 1. LOAD MASTER DATA
# ============================================================


master = pd.read_csv("data/master_final.csv")


print("\nDataset Shape:")
print(master.shape)


print("\nNumber of Districts:")
print(master["District"].nunique())



# ============================================================
# 2. RANDOM FOREST - BED CAPACITY PREDICTION
# ============================================================


print("\n" + "=" * 60)
print("RANDOM FOREST - HOSPITAL BED CAPACITY PREDICTION")
print("=" * 60)



# Features used to predict hospital beds
```

```
features = [
    "Population",
    "Total_Facilities",
    "Doctors in all Hospitals",
    "Health Sub-Centres",
    "Primary Health Centres",
    "Community Health Centres",
    "Area Hospitals",
    "District Hospitals",
    "Teaching Hospitals",
    "Ayurveda Hospitals (incl. Dispensaries)",
    "Homeopathic Hospitals (incl. Dispensaries)",
    "Unani Hospitals (incl. Dispensaries)",
    "Naturopathy Hospitals (incl. Dispensaries)"
]

target = "Beds in all Hospitals"



# Remove rows with missing values
ml_data = master[features + [target]].dropna()

X = ml_data[features]
y = ml_data[target]



print("\nFeatures used:")
for feature in features:
    print("-", feature)

print("\nTarget:")
print("-", target)



# ============================================================
# 3. TRAIN / TEST SPLIT
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)



print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))



# ============================================================
# 4. RANDOM FOREST MODEL
# ============================================================

model = RandomForestRegressor(
    n_estimators=300,
    random_state=42
)

model.fit(X_train, y_train)



# ============================================================
# 5. PREDICTIONS
# ============================================================

y_pred = model.predict(X_test)
```

```
# ============================================================
# 6. MODEL EVALUATION
# ============================================================

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)



print("\nModel Performance:")
print("-----------------------------")
print("MAE :", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R²  :", round(r2, 4))



# ============================================================
# 7. ACTUAL VS PREDICTED
# ============================================================

results = pd.DataFrame({
    "Actual_Beds": y_test.values,
    "Predicted_Beds": y_pred
})

print("\nActual vs Predicted:")
print(results)



# ============================================================
# 8. FEATURE IMPORTANCE
# ============================================================

importance = pd.DataFrame({
    "Feature": features,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    "Importance",
    ascending=False
)

print("\nFeature Importance:")
print(importance)



# ============================================================
# 9. FEATURE IMPORTANCE GRAPH
# ============================================================


plt.figure(figsize=(10, 7))

plt.barh(
    importance["Feature"],
    importance["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Random Forest Feature Importance")


plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```

```
# ============================================================
# 10. ACTUAL VS PREDICTED GRAPH
# ============================================================

plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    y_pred
)

plt.xlabel("Actual Hospital Beds")
plt.ylabel("Predicted Hospital Beds")
plt.title("Actual vs Predicted Hospital Bed Capacity")

plt.tight_layout()
plt.show()



# ============================================================
# 11. K-MEANS CLUSTERING
# ============================================================

print("\n" + "=" * 60)
print("K-MEANS - DISTRICT INFRASTRUCTURE CLUSTERING")
print("=" * 60)



# Create infrastructure indicators
cluster_data = master[
    [
        "District",
        "Beds_per_1000",
        "Doctors_per_100000",
        "Facilities_per_100000",
        "Rural_Facility_Percent"
    ]
].dropna().copy()



# Features for clustering
cluster_features = [
    "Beds_per_1000",
    "Doctors_per_100000",
    "Facilities_per_100000",
    "Rural_Facility_Percent"
]

X_cluster = cluster_data[cluster_features]



# ============================================================
# 12. STANDARDIZE DATA
# ============================================================

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_cluster)



# ============================================================
# 13. ELBOW METHOD
# ============================================================

inertia = []
```

```
for k in range(2, 7):

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    inertia.append(kmeans.inertia_)



plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 7),
    inertia,
    marker="o"
)

plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("K-Means Elbow Method")

plt.tight_layout()
plt.show()



# ============================================================
# 14. CREATE 3 DISTRICT CLUSTERS
# ============================================================

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

cluster_data["Cluster"] = kmeans.fit_predict(X_scaled)



# ============================================================
# 15. DISPLAY CLUSTERS
# ============================================================

print("\nDistrict Clusters:")

print(
    cluster_data[
        [
            "District",
            "Beds_per_1000",
            "Doctors_per_100000",
            "Facilities_per_100000",
            "Rural_Facility_Percent",
            "Cluster"
        ]
    ].sort_values("Cluster")
)



# ============================================================
# 16. CLUSTER PROFILES
# ============================================================

cluster_profile = cluster_data.groupby("Cluster")[
```

```
    cluster_features
].mean()

print("\nCluster Profiles:")
print(cluster_profile)



# ============================================================
# 17. SAVE CLUSTER RESULTS
# ============================================================

cluster_data.to_csv(
    "data/district_clusters.csv",
    index=False
)

print("\nSaved:")
print("data/district_clusters.csv")



# ============================================================
# 18. ADD CLUSTER TO MASTER DATA
# ============================================================

master_with_clusters = master.merge(
    cluster_data[["District", "Cluster"]],
    on="District",
    how="left"
)

master_with_clusters.to_csv(
    "data/master_ml.csv",
    index=False
)

print("data/master_ml.csv")



# ============================================================
# 19. FINAL MESSAGE
# ============================================================

print("\n" + "=" * 60)
print("MACHINE LEARNING COMPLETE")
print("=" * 60)
```

Appendix B – How to Run

B.1 Running the project

1. Install the libraries: pip install pandas numpy matplotlib seaborn plotly scikit-learn geopy streamlit
2. Keep the data folder (and data_old for the raw register) next to the scripts.
3. Run python build_data.py, then python eda.py, then python ml.py to rebuild the master table, charts and clusters.
4. Start the dashboard with streamlit run app.py and open http://localhost:8501 in a browser.